

ServiceNow GlideAjax: Synchronous (Sync) vs. Asynchronous (Async)

One of the most critical concepts in ServiceNow scripting is understanding the difference between Synchronous (Sync) and Asynchronous (Async) execution when using GlideAjax in `onChange` Client Scripts.

This guide breaks down these concepts using layman analogies, visual diagrams, and side-by-side code comparisons so you can easily understand which one to use and why.

1. The Analogy: Understanding Sync vs. Async

Synchronous (Sync) is like a Phone Call

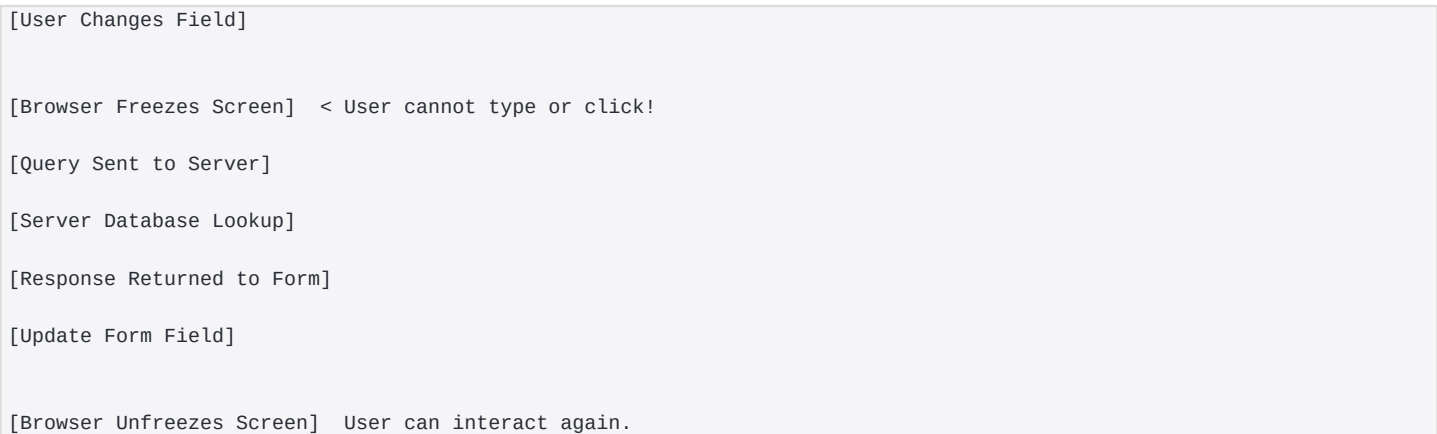
- How it works: You dial a phone number. You must keep the phone held to your ear, waiting for the other person to answer. While you are waiting, your hands and attention are fully occupied; you cannot do anything else (like wash dishes or type an email) until the call ends.
- In ServiceNow: When a Client Script runs a Synchronous GlideAjax call, the web browser freezes the entire form. The user cannot click checkboxes, type in text fields, or save the form. The page remains completely locked and unresponsive until the database query completes.

Asynchronous (Async) is like a Text Message

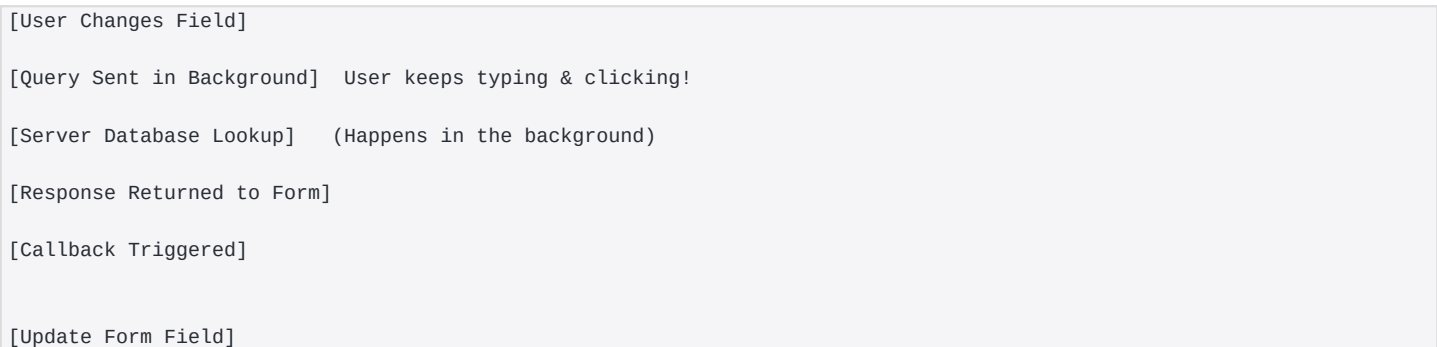
- How it works: You type a text message and hit "Send". You immediately put the phone back in your pocket and continue washing dishes, typing code, or walking. When the other person replies, your phone buzzes (this notification is the Callback Function). You stop what you are doing, read the reply, and take action.
- In ServiceNow: When an Asynchronous GlideAjax call runs, the browser sends the query to the server in the background. The user can immediately keep typing description logs, selecting CIs, and filling out the form. When the database finishes looking up the information, it calls a dedicated block of code (the Callback Function) to update the form fields automatically.

2. Dynamic Execution Flow Diagrams

Synchronous Flow (Blocks User)



Asynchronous Flow (Background Task)



3. Which One is Used and Why?

Always use Asynchronous (Async) GlideAjax.

Why Asynchronous (Async) is the standard:

1. User Experience (UX): No page freezing or loading lag. The browser remains active and smooth.
2. Platform Constraints: Synchronous calls (like `getXMLWait()`) are not supported in Service Portal, Mobile App environments, or Scoped Applications. Running them will throw javascript runtime errors.
3. Performance Standards: ServiceNow instance health checks and certifiers will flag synchronous client scripts as critical performance violations.

4. Practical Implementation: Step-by-Step Code

Let's build a real-world scenario: When the Caller field changes on an Incident form, lookup their Email Address from the database and populate a custom field ``u_caller_email``.

The Server Code: Script Include (Matches both Sync and Async)

Create a Script Include. It must be marked as Client Callable and inherit from ``AbstractAjaxProcessor``.

- Name: ``GetUserInfo``
- Client Callable: True (checked)
- Script:

```
var GetUserInfo = Class.create();
GetUserInfo.prototype = Object.extend(Object.prototype, {

  // This method is called from the client
  getEmailAddress: function() {
    // Read parameters sent from Client Script
    var userId = this.getParameter('sysparm_user_id');

    var userGR = new GlideRecord('sys_user');
    if (userGR.get(userId)) {
      // Return email string back to the client
      return userGR.getValue('email');
    }
    return '';
  },

  type: 'GetUserInfo'
});
```

The Client Code: Side-by-Side Comparison

The WRONG Way: Synchronous (Sync)

This method freezes the browser and will fail on Service Portal/Mobile.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }

  var ga = new GlideAjax('GetUserInfo');
  ga.addParam('sysparm_name', 'getEmailAddress');
  ga.addParam('sysparm_user_id', newValue);

  // getXMLWait() pauses the browser thread until database returns data
  ga.getXMLWait();
}
```

```
// Read response directly after wait (Sync)
var email = ga.getAnswer();
g_form.setValue('u_caller_email', email);
}
```

The RIGHT Way: Asynchronous (Async)

This method runs smoothly in the background, does not freeze pages, and is fully compatible with Service Portal.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    // 1. Instantiate the GlideAjax class pointing to the Script Include
    var ga = new GlideAjax('GetUserInfo');
    // 2. Specify the method to call on the server
    ga.addParam('sysparm_name', 'getEmailAddress');
    // 3. Send parameters (e.g. Selected User Sys ID)
    ga.addParam('sysparm_user_id', newValue);

    // 4. Send query asynchronously. Pass the CALLBACK function.
    // getXMLAnswer immediately releases browser focus so user can type.
    ga.getXMLAnswer(handleEmailResponse);
}

// 5. The Callback Function: runs automatically when the database answers.
function handleEmailResponse(answer) {
    // Populate the email address on the form
    g_form.setValue('u_caller_email', answer);
}
```

Key Takeaway Checklist

- Use `getXMLAnswer(callbackFunction)` to execute Asynchronous lookups.
- Write a separate `callbackFunction(response)` to handle the data once returned.
- Never use `getXMLWait()` or synchronous loops; they degrade execution performance.
- In `onChange` scripts, always verify `isLoading || newValue === ''` conditions to prevent recursive trigger runs.